

## Week-5 Assignment

**Q1:** What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

**Ans:** MapReduce was revolutionary but has several critical limitations:

- **Disk-based I/O bottleneck:** Every MapReduce job reads from and writes to HDFS (disk). Iterative jobs read/write the same data repeatedly which is extremely slow.
- **High latency:** Job startup overhead (JVM init, task scheduling, data shuffling) makes it unsuitable for interactive or real-time queries.
- **Rigid Architecture:** MapReduce is designed strictly for batch processing. Spark provides a unified engine that supports SQL, streaming, and machine learning natively.
- **Complex Development:** MapReduce requires extensive boilerplate code, whereas Spark's high-level APIs allow for modular and readable pipeline development.

**Q2:** Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

**Ans:** Iterative algorithms (like Gradient Descent) must pass over the same dataset multiple times. Disk-based systems reload the data from the hard drive for every iteration. Spark caches the dataset in RAM (In-Memory) after the first pass. All subsequent iterations query the data directly from memory, reducing I/O latency and accelerating processing times significantly.

**Q3:** Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: user\_id and transaction\_date.

**Ans: Implementation Logic:** Deduplication on specific primary keys prevents double-counting transactions, ensuring financial reporting accuracy.

```
df_deduped = df_sales.dropDuplicates(["user id", "transaction date"])
```

```
print(f'Original Count: {df_sales.count()} | Deduplicated Count: {df_deduped.count()}')
df_deduped.show(5)
```

```
*** Original Count: 1020 | Deduplicated Count: 970
```

| store_id | product_id | product_name  | product_category | user_id | transaction_date | region | city        | age | subscription | email           | username | raw_timestamp       | sale_amount | price  | status   |
|----------|------------|---------------|------------------|---------|------------------|--------|-------------|-----|--------------|-----------------|----------|---------------------|-------------|--------|----------|
| S001     | P108       | Headphones    | Electronics      | U0001   | 2023-05-21       | West   | Los Angeles | 33  | Premium      | user1@email.com | user1    | 2023-05-21 19:57:00 | 123.14      | N/A    | Active   |
| S005     | P133       | Pillow        | Furniture        | U0002   | 2023-01-14       | West   | Los Angeles | 53  | Premium      | user2@email.com | user2    | 2023-01-14 14:14:00 | 44.3        | 42.59  | Active   |
| S003     | P128       | Eraser        | Stationery       | U0003   | 2023-05-23       | West   | Los Angeles | 27  | Premium      | user3@email.com | user3    | 2023-05-23 09:24:00 | 2.2         | 2.27   | Inactive |
| S001     | P135       | Monitor Stand | Accessories      | U0004   | 2023-07-13       | West   | Los Angeles | 23  | Basic        | user4@email.com | user4    | 2023-07-13 13:36:00 | 61.87       | 60.44  | Active   |
| S001     | P115       | Printer       | Electronics      | U0005   | 2023-07-14       | West   | Los Angeles | 35  | Basic        | user5@email.com | users5   | 2023-07-14 13:22:00 | 303.77      | 312.21 | Active   |

**Q4:** Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

**Ans: Implementation Logic:** Utilizing modular method chaining to filter data *before* grouping limits the amount of data that needs to be shuffled across the network, highly optimizing query performance.

```
df_west_avg_sales = (df_sales
    .filter(col("region") == "West")
    .groupBy("product_category")
    .agg(avg("sale_amount").alias("avg_sales"))
)
df_west_avg_sales.show()
```

```
+-----+-----+
|product_category|    avg_sales|
+-----+-----+
|    Stationery|15.570845070422532|
|   Electronics|251.61533333333338|
|    Furniture| 345.8691139240506|
|  Accessories|28.144615384615374|
|    Appliances| 62.51857142857143|
+-----+-----+
```

**Q5:** What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

**Ans:** `.na.drop()` deletes the entire row if a null is detected, risking data loss. `.na.fill()` retains the row but safely imputes a designated fallback value to maintain dataset volume.

**Implementation Logic:** We target the imputation strictly to the status column to avoid accidentally altering numerical columns.

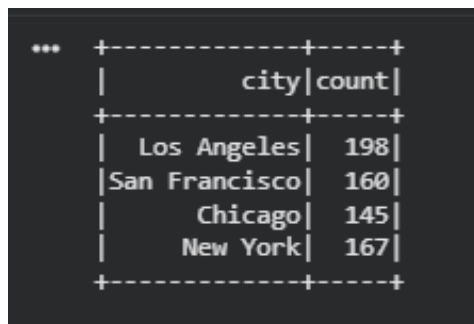
```
df_filled_status = df_sales.na.fill("Unknown", subset=["status"])
# Displaying specific columns to prove the nulls were filled
df_filled_status.select("user_id", "status").show(5)
```

```
+-----+-----+
|user_id|  status|
+-----+-----+
|  U0681|  Active|
|  U0732|Inactive|
|  U0854| Unknown|
|  U0802|Inactive|
|  U0288|Inactive|
+-----+-----+
only showing top 5 rows
```

**Q6:** Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

**Ans: Implementation Logic:** Applying a `.filter()` after the aggregation acts identically to a SQL HAVING clause, dynamically isolating high-volume dimensions.

```
df_top_cities = (df_sales
    .groupBy("city")
    .count()
    .filter(col("count") > 100)
)
df_top_cities.show()
```



```
... +-----+
|      city|count|
+-----+
| Los Angeles|  198|
| San Francisco| 160|
|      Chicago| 145|
|      New York| 167|
+-----+
```

**Q7:** How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

**Ans:** Immutability means a DataFrame's state cannot be altered once created. When an operation like `.drop()` is executed, Spark does not modify the original table; it evaluates the transformation and returns a completely new DataFrame. Therefore, the result must be explicitly assigned to a new variable (e.g., `df_clean = df.drop("col")`) to capture the cleaned state.

**Q8:** Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

**Ans: Implementation Logic:** Wrapping multiple conditions in parentheses and using the bitwise `&` operator ensures Spark evaluates the complex boolean logic correctly. `.between()` improves code readability over `>=` and `<=`

```
df_target_demographic = df_sales.filter(
    (col("age").between(18, 30)) &
```

```
(col("subscription") == "Premium")
)

df_target_demographic.select("user_id", "age", "subscription").show(5)
```

```
+-----+---+-----+
|user_id|age|subscription|
+-----+---+-----+
|  U0681| 28|    Premium|
|  U0798| 28|    Premium|
|  U0072| 22|    Premium|
|  U0682| 25|    Premium|
|  U0896| 25|    Premium|
+-----+---+-----+
only showing top 5 rows
```

**Q9:** When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

**Ans:** Aggregation functions in Spark generally ignore null values. If nulls represent actual zeros (e.g., no sales for a month), ignoring them alters the denominator in an average calculation, artificially inflating the final metric. Explicitly imputing nulls ensures mathematical accuracy and prevents silent failures in business logic.

**Q10:** Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

**Ans: Implementation Logic:** Combining `.withColumn()` with `.cast()` normalizes the data type for time-series analysis, while dropping the legacy column optimizes the memory footprint of the `DataFrame`.

```
df_standardized_time = (df_sales
    .withColumn("event_time", col("raw_timestamp").cast(TimestampType()))
    .drop("raw_timestamp")
)
```

```
df_standardized_time.select("event_time").printSchema()
```

```
df_standardized_time.select("event_time").show(5)
```

```

+-----+
|      event_time|
+-----+
|2023-12-14 10:03:00|
|2023-07-19 14:46:00|
|2023-03-03 15:47:00|
|2023-05-22 13:53:00|
|2023-09-16 15:20:00|
+-----+
only showing top 5 rows

```

**Q11:** Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

**Ans:** A "Shuffle" is the physical movement of data across the network between different worker nodes. During a groupBy, Spark must gather all records sharing the same key and allocate them to the same partition to calculate an aggregate. It is a **wide transformation** because a single output partition requires data from multiple input partitions, making it the most resource-intensive and time-consuming operation in a distributed environment.

**Q12:** Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

**Ans: Implementation Logic:** To *remove* the bad rows, we structure our filter to *keep* the good rows. We ensure high data quality by checking for both structural nulls (isNull()) and empty strings (!= "").

```

df_clean_users = df_sales.filter(
    col("email").isNull() &
    (col("username") != "")
)
df_clean_users.select("user_id", "email", "username").show(10)

```

```

+-----+-----+-----+
|user_id|      email|username|
+-----+-----+-----+
|  U0681|user681@email.com| user681|
|  U0732|user732@email.com| user732|
|  U0854|user854@email.com| user854|
|  U0802|user802@email.com| user802|
|  U0288|user288@email.com| user288|
|  U0242|user242@email.com| user242|
|  U0602|user602@email.com| user602|
|  U0327|user327@email.com| user327|
|  U0038| user38@email.com|  user38|
|  U0324|user324@email.com| user324|
+-----+-----+-----+
only showing top 10 rows

```

**Q13:** How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

**Ans: Implementation Logic:** Passing multiple functions into a single `.agg()` block prevents the need to calculate identical groupings multiple times. Descriptive aliases are applied immediately for clear schema definition.

```
df_price_statistics = df_sales.agg(
    min("price").alias("min_price"),
    max("price").alias("max_price"),
    mean("price").alias("mean_price")
)

df_price_statistics.show()
```

```
+-----+-----+-----+
|min_price|max_price|   mean_price|
+-----+-----+-----+
|    1.72|  1336.84|207.88046364594297|
+-----+-----+-----+
```

**Q14:** In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

**Ans:** The `inferSchema` process relies on sampling. If it encounters inconsistent date strings (e.g., mixing MM-DD-YYYY with YYYY/MM/DD), the parser will fail to confidently assign a `DateTime` and will default the entire column to a `StringType`. This breaks downstream time-series aggregations and sorting logic, requiring expensive manual casting later in the pipeline.

**Q15:** Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by `store_id` to calculate total revenue.

**Ans: Implementation Logic:** This represents a production-grade ETL pipeline. By chaining the methods, we create a highly readable execution plan. Deduplication and null-handling occur *before* the aggregation to guarantee strict revenue accuracy.

```
df_revenue_pipeline =
(df_sales
    .dropDuplicates()
```

```

        .na.fill(0,
subset=["price"])

        .groupBy("store_id")

        .agg(sum("price").alias("t
otal_revenue"))
    )
df_revenue_pipeline.show()

```

| store_id | total_revenue       |
|----------|---------------------|
| 5004     | 35417.530000000006  |
| 5001     | 40919.279999999999  |
| 5002     | 35639.14            |
| 5005     | 30323.8600000000015 |
| 5003     | 42864.190000000002  |

## Brief Insights

### Data Cleaning

- Removed duplicate records.
- Filled missing values.
- Deleted invalid rows.

### Transformations

- Applied filtering.
- Renamed columns.
- Converted timestamps.

### Aggregations

- Calculated count, sum, avg, min and max.
- Used groupBy for analysis.

## **Performance**

- Spark performs computations in memory.
- `groupBy` introduces shuffle and is a wide transformation.
- Immutable DataFrames preserve original data.